Oracle design and complexity analysis for a Grover-based quantum maze solver

Advithi Baddam[1,5], Julia Pryor[2,5], Revati Tambe[3,5], and Nathan Tao[4,5]

[1]Allen High School, Allen, TX, 75002, USA
[2]The Kinkaid School, Houston, TX, 77024, USA
[3]Homestead High School, Cupertino, CA, 95014, USA
[4]Scarsdale Senior High School, Scarsdale, NY, 10583, USA
[5]All co-authors have made an equal contribution to the work.

We present the design and analysis of a quantum algorithm for solving perfect mazes using Grover's search. The algorithm operates on a superposition of all possible paths of a fixed length L, encoded in a quantum register. We developed a modular oracle, the `FitnessGate`, to evaluate all paths in parallel, validating them against maze walls and using a Manhattan distance-based metric to determine their fitness. Through classical simulation, the algorithm successfully solved 95.90% of 2×2 mazes. However, for 3×3 mazes, the solve rate dropped to 43.04% while simulation time increased dramatically, highlighting significant scaling challenges. Our complexity analysis reveals the primary source of this challenge: the algorithm's runtime scales as $\mathcal{O}(2^L \cdot \mathbf{Poly}(L, n))$, which is exponential in the path length L. This confirms that searching the state space of all possible paths does not offer a quantum advantage over efficient polynomial-time classical solvers. Our work underscores the critical importance of problem representation in quantum algorithm design and motivates the exploration of alternative methods, such as quantum walks, for future research.

Pathfinding algorithms, which identify optimal routes between nodes in a graph, are fundamental to computational science, with applications ranging from network routing to logistical optimization and robotics [1]. Maze-solving represents a canonical instance of the pathfinding problem, providing a structured yet challenging environment to develop and analyze novel algorithmic approaches. While classical algorithms such as Breadth-First Search (BFS) can solve certain classes of mazes efficiently, the prospect of quantum computation offers new paradigms for tackling search problems, exemplified by Grover's algorithm, which provides a quadratic speedup for unstructured search [2].

Building on conceptual ideas from prior Grover-based quantum path finding research [3], this work investigates the application of a Grover-based quantum algorithm to the problem of solving a perfect maze. A perfect maze is a grid-based graph with no closed loops, guaranteeing a single unique path between any two points. Our algorithm operates by encoding a superposition of all possible paths of a fixed length L into a quantum register. We designed and implemented a modular quantum oracle, the `FitnessGate`, which validates each path in the superposition by checking for collisions with maze walls and evaluates its proximity to the designated end-point.

The primary contribution of this work is therefore twofold: first, the design and implementation of a modular quantum oracle for path validation, and second, a rigorous complexity analysis to formally evaluate its performance against established classical algorithms. This analysis allows us to critically assess the viability of the path-representation model for demonstrating a quantum advantage in this problem space.

1 Maze Generation

To provide a consistent and valid problem space for our algorithm, we procedurally generate a unique `n x n` perfect maze for each simulation. The generation is performed using the Recursive

Backtracker algorithm, a randomized variant of the depth-first search (DFS) algorithm [5].

The process begins by creating a grid of `n x n` cells, where each cell is initially enclosed by four walls. A random cell is chosen as the starting point, marked as visited, and pushed onto a stack. The algorithm then iteratively inspects the cell at the top of the stack. If this cell has any unvisited neighbors, one is chosen at random, the wall between the two cells is removed, and the neighbor is pushed onto the stack. If the current cell has no unvisited neighbors, signifying a dead end, the algorithm backtracks by popping from the stack. The process terminates when the stack is empty, at which point a fully connected, perfect maze has been carved from the initial grid. This method guarantees a perfect maze structure, which is essential for defining a clear and unambiguous solution path for our algorithm to find.

2 Defining the Path Register

The foundational element of our quantum algorithm is the representation of a possible solution path. Unlike classical algorithms that must explore one path at a time, our approach encodes a superposition of all possible paths into a single quantum register. This `path_register` is a sequence of quantum bits, or qubits for short, that represents a series of directional moves throughout the maze.

2.1 Encoding directions

A path is defined as a sequence of directional moves of a fixed length, L. To represent these moves, we can first encode the four cardinal directions into a 2-qubit computational basis:

- North (N) is represented by the state $|00\rangle$
- East (E) is represented by the state $|01\rangle$
- South (S) is represented by the state $|10\rangle$
- West (W) is represented by the state $|11\rangle$

The primary register, the `path_register`, requires $2L$ qubits to represent a full path of length L. The algorithm begins by creating a uniform superposition of all possible paths. This is achieved by applying a Hadamard gate to each of the $2L$ qubits in the `path_register`. This single operation prepares a state representing 2^{2L} (or 4^L) possible paths, which accounts for the exponentially large search space for our algorithm.

2.2 Register allocation

The complete quantum circuit requires several registers to execute the algorithm:

- Path register (`path_register`): This is the primary register representing the sequence of moves. For a path of fixed length L, this register requires $2L$ qubits, as each of the L directional steps is encoded by 2 qubits.
- Coordinate registers (`xc_register`, `yc_register`): These registers track the current (x, y) position of a path as it is simulated. The number of qubits required for each coordinate register is determined by the maze dimension n, specifically $b = ceil(log(n))$, as b qubits can represent integers up to $2^b - 1$.
- Ancilla registers: Several ancillary qubits are required for intermediate computations. These are used as output bits for oracles (e.g., in `IsClosedGate`), as carry bits in quantum arithmetic, and as target qubits for the phase kickback mechanism in Grover's algorithm.

2.3 Space cost

The choice of the path length L is a critical parameter. A small L may not be long enough to reach the end of the maze, while a large L dramatically increases both the qubit requirement ($2L$) and the size of the search space (4^L), which directly and exponentially impacts the algorithm's overall runtime. Therefore, the total qubit count, or space cost, is dominated by the path length L. In a worst-case scenario, such as a maze designed with a long path that visits every single room, the required L can be as large as the total number of rooms, and approach n^2.

3 Fitness operator

A maze solving algorithm must distinguish between good and bad paths. In our approach, inspired by (citation to original), each path is assigned a fitness value. A path's fitness is its

estimated distance to the maze exit (lower fitnesses are generally better). We adapt (reference to original)'s approach with:

- A more cohesive and developed quantum path simulation
- A more efficient and realistic distance calculation

3.1 Quantum path simulation

Let $|P\rangle$ be our path register, currently in a superposition of all valid and invalid paths of qubit count $2l$ (corresponding to l directions). Let $b(x) = \lceil \log_2 (x+1) \rceil$ describe the number of bits needed to represent a number. We will commonly refer to $b(s)$, the number of bits to represent our maze's side length. Define "individual" as a single state within $|P\rangle$. $|P\rangle$ contains 4^l individuals Define "stop point" as an individual's ending cell. This stop point is achieved when all directions in the path are processed. Reasonably, to calculate an individual's fitness (the estimated distance from the stop point to the end), we must first, in some form, identify this stop point. In our algorithm, this is achieved through simulation. Let $|x, y\rangle$ represent the stop point of each path (these are our 'xc' and 'yc' registers). We aim to entangle this register with $|P\rangle$ such that each individual corresponds to its stop point. The following procedure describes a possible process to achieve this entanglement: Define a maze coordinate system as zero-indexed (column, row), with $(0, 0)$ as top-left and $(s-1, s-1)$ as bottom-right. Let (c_i, r_i) represent the maze's starting cell; (c_f, r_f), the ending cell.

1. Initialize $|x, y\rangle$ as the maze's starting point, applying:

$$\bigotimes_{k=0}^{b(s)-1} X^{c_{i,k}} |x\rangle \bigotimes_{k=0}^{b(s)-1} X^{r_{i,k}} |y\rangle$$

   where $c_{i,k}$ is the $k+1$-th least significant bit in c_i, and X and Y are the Pauli-X and Pauli-Y gates. With a very similar process, initialize $|f\rangle$ as the maze's ending point.

2. For i from 0 to $l-1$,

   - The $i+1$-th directions (moves) are represented by the two-qubit ket $|p_i\rangle$.
   - If a move will intersect with (would go through) a wall, we will flip an ancilla (initially $|0\rangle$), $|A_{1,i}\rangle$. This is performed by the *IsClosed* oracle, which takes in $|x, y, p_i, 0\rangle$ and returns $|x, y, p_i, 0\rangle$ if moving $|x, y\rangle$ by $|p_i\rangle$ would not intersect a wall, and $|x, y, p_i, 1\rangle$ otherwise.
   - If, in a given individual, $|x, y\rangle$ has the same bit values as $|f\rangle$ (and theoretically, $|x, y\rangle$ is not in superposition for a fixed individual), then we flip an ancilla (initially $|0\rangle$), $|A_{2,i}\rangle$. This is performed by the *IsEnd* oracle. The purpose of this move is to stop a path from continuing if the maze exit is reached.
   - Execute the *UpdateRoomGate* if $|A_{1,i}\rangle$ and $|A_{2,i}\rangle$ are $|0\rangle$, which advances $|x, y\rangle$ based on $|p_i\rangle$

3.2 IsClosed oracle

The `Is Closed` Oracle flips an ancilla from $|0\rangle$ to $|1\rangle$ if a move would go through a maze wall (in other words, the route for the move is closed). A brute-force circuit for the `Is Closed` Oracle is described in Figure 1.

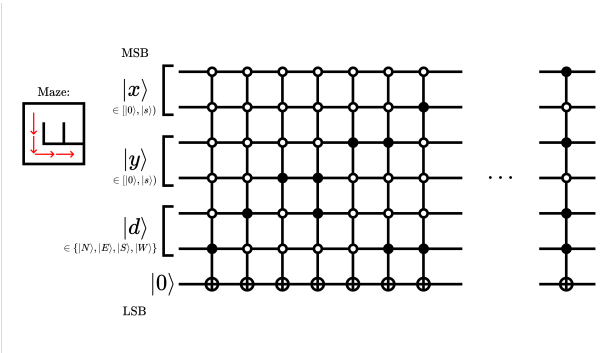

Figure 1: An Is Closed Oracle composed of $2b(s)+2$ toffolis, $O(n^2)$ times

If the x, y, and p_i registers all correspond to a valid move for a given quantum state, this circuit will utilize Pauli-X gates with $2b(s)+2$ controls ($b(s)$ for each coordinate, and 2 for the direction). The time complexity is $O(n^2 \log_2 n)$: n^2 for the number of Pauli-X gates, and $\log_2 n$ for the cost of checking each gate's controls.

For classical simulators, there may be a speedup, inspired by a binary search tree. The new diagram is illustrated in Figure 2.

Each qubit may receive a zero or one control, which activates sub-gates of similar prop-

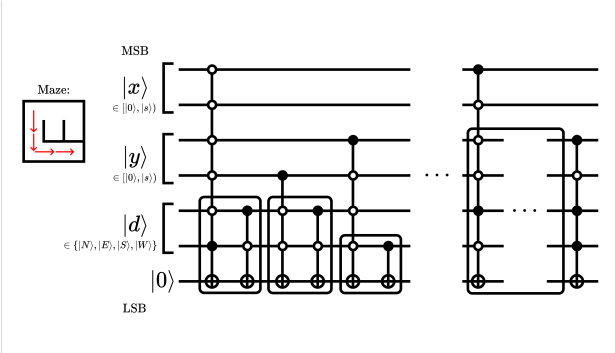

Figure 2: A binary search tree optimization for some classical-based quantum simulators

erties. The final ancillary qubit eventually might be flipped by a CX gate. As such, we *may* reduce repetition on some classical computer-based simulators. If optimization is achieved, the resulting time complexity is likely $O((\log_2 n)^2)$. No optimization is likely achieved on quantum computers, as all the gates will be applied.

If optimized solutions to maze solving are to be found with similarly-constructed algorithms in future research, then the `Is Closed` Oracle is the primary area for improvement.

3.3 Is End oracle

The *Is End* Oracle checks if the current position in the iteration, $|x, y\rangle$, is equal to the maze end position, represented in the register $|F_X, F_Y\rangle$. *Is End* implementations using a circuit that encodes the maze exit position and implementations that check for equality in states (using the XOR, then control method described in the *Is Closed* Oracle section of this paper) share similar effectiveness.

3.4 Update room gate

The Update Room Gate, U, shifts the path's current position, $|x, y\rangle$, by its move this iteration, $|p_i\rangle$, thus changing the current position to advance one move. Mathematically, it is defined as follows:

$$
\begin{aligned}
U\,|p_i, x, y\rangle = \\
|\mathrm{N}\rangle\langle\mathrm{N}| \otimes |x\rangle \otimes \left(\sum_y |y-1\rangle\langle y|\right) + \\
|\mathrm{E}\rangle\langle\mathrm{E}| \otimes \left(\sum_x |x+1\rangle\langle x|\right) \otimes |y\rangle + \\
|\mathrm{S}\rangle\langle\mathrm{S}| \otimes |x\rangle \otimes \left(\sum_y |y+1\rangle\langle y|\right) + \\
|\mathrm{W}\rangle\langle\mathrm{W}| \otimes \left(\sum_x |x-1\rangle\langle x|\right) \otimes |y\rangle.
\end{aligned}
$$

We apply controlled add and subtract gates that update our current position based on our direction.

3.5 Manhattan distance

Formally, the Manhattan distance between points (y_1, x_1) and (y_2, x_2) is $|y_1 - y_2| + |x_1 - x_2|$. Defining distance in this manner, as opposed to the more Euclidean definition of distance, reduces space requirements (since Euclidean distance is defined as $\sqrt{|y_1 - y_2|^2 + |x_1 - x_2|^2}$ and requires more space to calculate and store the term inside in the square root) but more importantly, provides a more accurate representation of distance in the grid with respect to motions that can only be done in cardinal directions. For example, if using a Euclidean distance system, the distance between rooms separated by a move south and a move east would be described as $\sqrt{2}$ and thus shorter than the distance between rooms separated by two moves to the south, which would be 2. However, since under our notion of a maze an individual moving through the maze can only make moves in the cardinal directions, the steps required to reach one room from the other in both of the aforementioned cases is 2 and the individual in the maze should consider each path equally slow. Because Manhattan distance requires less resources and reflects the distance between rooms in respect to the number of steps, in cardinal directions, needed to be taken to travel between them, Manhattan distance is preferred.

The resulting Manhattan distance becomes our fitness (stored in the fitness register).

4 Fitness threshold

A fitness thresholder gate is an oracle designed to take an input of the path and its corresponding fitness, represented in the qubit registers, as well as an ancilla bit set up in the $|1\rangle$ state, and will output both the ancilla bit and the combined path-fitness register flipped along the Z-axis if the input fitness register represents a fitness that is lower than or equal to a certain threshold.

Constructing an oracle that meets the specifications that a certain path-fitness register has a fitness of 0, i.e. that it reaches the goal, can be done by using a Pauli Z gate that is zero controlled on all of the fitness register applied to the ancilla bit; as a result of phase kickback, this will cause the path-fitness register to flip along the Z-axis as well.

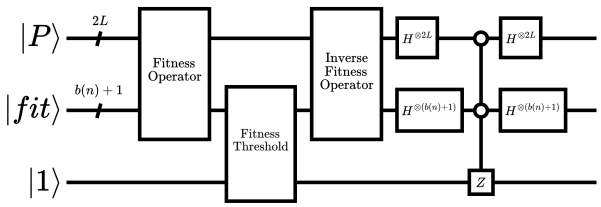

Figure 3: The fitness thresholding in the context of Grover's search

Furthermore, any fitness thresholder gate designed to threshold fitness values below or equal to a $2^k - 1$ value can be constructed simply by applying the Z gate to the ancilla controlled on all the fitness register except the first k least significant bits of that register; by definition, k here must be an integer, but fitness thresholders that work on a greater range of threshold values can be constructed, albeit with a greater number of controlled gates.

5 Grover-based repeatable search

To ensure a high probability of measuring a successful path, we implement a modified version of Grover's search algorithm. In the search algorithm that phase-flips a specific bitstring and then applies a diffusion operator that can be constructed by applying Hadamards to each qubit, including a multi-controlled Pauli-Z gate applied to an ancilla qubit (in the $|1\rangle$ state), zero controlled on all the qubits, and then once again applying Hadamards to each qubit.

Grover shows that the diffusion operator here acts in a way that flips the coefficients of the quantum state around their average, thus increasing the amplitude of measuring one bitstring in particular, if the coefficient of that bitstring had been negative before - precisely, that each iteration of the algorithm will increase the amplitude of the specific bitstring on the order of $O\left(\frac{1}{\sqrt{2^q}}\right)$ where q is the number of qubits used. Thus, the number of iterations necessary to maximize the probability of measuring that state is one of the order of $O(\sqrt{2^q})$, which is where the algorithm offers a speedup against a case in which a classical algorithm would have to check all 2^q (or a number around that) entries instead. Also notable is that if there exist multiple bitstrings flipped, provided that they are still in the minority, flipping the coefficients around the average will still increase the probability of those being measured, but of course only one can be measured at a time.

A complication arises in translating this algorithm to our case, which is that the use of the diffusion operator destroys the entanglement between the path register and its corresponding fitness register. This can be seen in that the diffusion operator employs the use of an array of Hadamard gates to each qubit in the path register, which will scramble information about the path and will not reconstruct it exactly through the second array of Hadamard gates because of the Pauli-Z gate between those gates, thus resulting in some path registers that are not ideal or completely invalid being marked with very favorable fitness, thus creating more error in the next iteration. This issue can be counteracted by only computing the fitness values around the fitness thresholder, where it is needed, and then running the fitness gate in adjoint for the diffusion operator and performing the same cycle on the next iteration. Because for our purposes we chose to end the repeated searches having calculated the fitness, the total repeated search becomes:

1. The fitness thresholder gate phase-flips the state(s) with adequate fitness.

2. The fitness gate is run in adjoint on the path and fitness registers, effectively uncomputing the fitness register.

3. Hadamard gates are applied to all of the path register.

4. A Multi-controlled Pauli-Z gate is applied to an ancilla, zero-controlled on all of the path register.

5. Hadamard gates are again applied to all of the path register.

6. The fitness gate is run on the path and fitness registers.

6 Time complexity analysis

Our analysis shows that the time complexity of the implemented quantum algorithm is **exponential** with respect to the path length L. The total runtime T can be expressed as the product of the number of Grover iterations and the gate complexity of the oracle executed in each iteration:

$$T = \mathcal{O}(\text{Iterations}) \times \mathcal{O}(\text{Oracle Complexity}) \quad (1)$$

We derive the final complexity, $T = \mathcal{O}(2^L \cdot \text{Poly}(L, n))$, by analyzing these two components.

6.1 Search space and Grover iterations

The algorithm searches the state space of all possible directional paths of a fixed length L. As each of the L steps can be one of four directions, the total number of paths, N, in the search space is $N = 4^L$.

Grover's algorithm requires approximately $\frac{\pi}{4}\sqrt{\frac{N}{M}}$ iterations to find a solution with high probability, where M is the number of valid, marked solutions [2]. Assuming at least one solution exists ($M \geq 1$), the number of iterations simplifies to $\mathcal{O}(\sqrt{N})$. Substituting the size of our search space, we find the number of required iterations to be:

$$\mathcal{O}(\sqrt{N}) = \mathcal{O}(\sqrt{4^L}) = \mathcal{O}(((2^2)^L)^{1/2}) = \mathcal{O}(2^L) \quad (2)$$

Thus, the number of times the `repeatable_search` function must be called grows exponentially with the path length L.

6.2 Oracle complexity

The oracle in our algorithm is the custom `FitnessGate`. Its complexity is determined by the number and type of gates within its definition. The `FitnessGate` contains a primary loop that iterates L times. Inside this loop, it appends several custom gates, with the most complex being the `IsClosedGate`.

The `IsClosedGate` itself is constructed via a classical loop that runs $\mathcal{O}(n^2)$ times (where n is the maze side length), adding a multi-controlled Toffoli gate (`mcx`) for each open path in the maze. The number of controls on these gates scales with the number of qubits required for the coordinates, which is $\mathcal{O}(\log n)$.

Therefore, the total gate complexity of a single application of the `FitnessGate` is polynomial in the path length L and the maze size n. We denote this as $\mathcal{O}(\text{Poly}(L, n))$.

6.3 Total complexity and classical comparison

By combining these two factors, we arrive at the total time complexity of the quantum algorithm:

$$T = \mathcal{O}(2^L \cdot \text{Poly}(L, n)) \quad (3)$$

To put this in context, we compare it to the standard classical algorithm for solving a perfect maze: a Breadth-First Search (BFS). The time complexity of a classical search on a grid graph is $\mathcal{O}(V + E)$, where V is the number of vertices (n^2) and E is the number of edges. This simplifies to a **polynomial** complexity of $\mathcal{O}(n^2)$.

The comparison is clear: our quantum algorithm's runtime is exponential, while the classical algorithm's is polynomial. Therefore, despite employing a quantum search algorithm, this path-representation approach does not provide a quantum advantage over established classical methods for this specific problem.

7 Results

Tests of FIGMS are limited by classical simulation capabilities and qubit count restrictions. We achieved classically-simulated accuracy data on two input configurations:

(s, l) vs Time, Solve Rate, and End Dist.		
	Inputs	
	s=2, l=2	s=3, l=4
Trials	1072	79
Avg. Time (secs)	3.005	656.102
Avg. End Dist.	0.076	2.595
Pct. Solve	95.90%	43.04%

In the above figure, "Avg. End Dist." denotes the average distance of a path to the maze finish. For paths that reach the end, this value is 0. This distance to the end is calculated by simulating the minimum moves necessary to reach the maze exit, rather than using Manhattan Distance. This metric serves to more practically evaluate FIGMS's accuracy; lower values are better.

Accuracy among 2x2 mazes is relatively high. 95.90% of paths resolved at the maze exit; those that did not emerged an average of 1.864 moves away. This result demonstrates the possibility of accurate quantum maze-solving algorithms.

On the contrary, 3x3 accuracy was significantly lower. A major contributor to this decreased effectiveness was the low l value. Due to qubit count and memory restrictions on classical simulators, it remains an area of possible research to study the algorithm's effectiveness under larger l values. Considering that only 88.54% of perfect 3x3 mazes are solvable in 4 steps, our 43.04% result indicates that, where reaching the maze exit is possible, we find the solution approximately $\frac{43.04\%}{88.54\%} = 48.61\%$ of the time. Overall, executing FIGMS on a 3x3 maze with $s = 3, l = 4$ outputs a solution $4 - 2.595 = 1.405$ steps closer to the maze exit than to the maze start.

Apart from the drop in accuracy at higher dimensions. The primary fault of FIGMS, by far, is the time complexity. Although classical simulators amplify the algorithm's time complexity inefficiency (as opposed to running on quantum computers), resolving the vast gap between the three-second 2x2 solve and eleven-minute 3x3 solve remains an area of potential research.

8 Discussion

The goal in any innovation of a quantum algorithm is generally to solve a computational problem in a better—faster, in most cases—manner than is possible with classical computing methods, though the careful use of particular phenomena unique to quantum computers. The particular phenomena of quantum superposition is often considered especially promising for the realm of quantum computing in how it seems similar to parallel processing, since when a quantum gate is applied to a qubit in superposition it, in some sense, acts on both the $|0\rangle$ and $|1\rangle$ state of the qubit [6]. The algorithm we present in this paper is based on this idea and uses it to perform a task similar to quantum walk, analogous to recording every possible movement of a classically random walk in an equal superposition against each other, and then applying logic that checks for validity and success in the constructed paths. This processing treats the superposition similarly to a system of bitstrings for each qubit state being processed in parallel, which is both what makes the algorithm unique to quantum computers but also its largest weakness, since superposition is in reality a balance of possible states with certain probabilities of measuring one of them; and the process of amplifying the probability of measuring the desired state through a modified Grover's search process is much too slow for any practical use compared against better classical options.

To illustrate the issue of Grover's search implementation in our algorithm and quantum maze-solving algorithms like it, one can consider the classical analog which our algorithm provides a speedup over, which would be an algorithm that generates a massive list of all possible movements (in the 4 cardinal directions) from a starting point in the maze—this is 4^L possibilities, where L is the expected length of the shortest path to the exit—and searching through each one of them, removing those that do not adhere to the boundaries of the maze provided and taking note of the sets of the movements that solve the maze in a valid manner, then selecting the shortest solution out of those. Besides the time required to generate that list, only searching through and processing the paths constructed in that classical approach would be very slow, asymptotically on the order of $O(4^L)$ time complexity. As shown in the time complexity analysis, section 6, the quantum algorithm presented here can perform this search in $O(2^L)$ time complexity thanks to the implementation of Grover's search algorithm that has been shown to provide quantum supremacy over classical search algorithms that work on unstructured databases—in this case an unstructured list of possible paths to take through a maze.

This is, of course, a terribly inefficient way to solve a maze in comparison to methods like Dijkstra's algorithm or A* search which can solve maze problems in polynomial time with respect to the number of points in the maze, which is asymptotically much better than our algorithm

of exponential time complexity [7]. The most apparent and stark contrast that appears between fast maze-solving algorithms and the algorithm presented in this paper is the use of the maze's structure to the algorithm's benefit. As mentioned above, Grover's search provides a speedup against classical computers in unstructured search problems, and the algorithm we present here treats maze-solving as an unstructured search problem in regards to processing and selecting an ideal path. In contrast, the aforementioned fast maze-solving algorithms construct and revise their best-guess solution as they navigate through the maze using information they obtain from the steps they have taken previously, from using estimates of their position relative to the end point, or both, and do not bother arduously considering every possible set of motions. Thus with that in consideration, we suggest that any quantum algorithm that would demonstrate supremacy over those classical algorithms that can already solve mazes in polynomial time would have to amplify the probability of measuring the more optimal solutions during the generation of the paths through the maze and not by searching through the possible paths afterwards before considering what speedups to accomplish on the steps in between.

Besides the inefficiency of many grover-based solutions, another challenge exists in quantum maze representation. Whereas classical computers simply reference a two or three-dimensional array in $O(1)$ time, no known equivalent exists when in quantum superposition. Since, at some point in the program's execution, the paths may check nearly all the cells (or at least around $O(n)$ of them to retain the use of superposition), most such algorithms will also need to detect the "shape" of all current cells, where shape refers to the configuration of walls and openings. Consequently, since $\Omega(n)$ different positions may exist in superposition at once, operations executed must support $\Omega(n)$ cases. The general solution, as mentioned in our `Is Closed` Oracle, is to iterate through all cells-wall combinations (in other words, all possible next moves), flipping an ancilla if a move is invalid. Of course, this is slow. Future studies can analyze methods to simplify maze representation. Below, we propose two solutions, which may be used together. Both may result in decreased preprocessing efficiency, in return for potential quantum advantage in individual queries.

Solution 1: Reduced Checks

A maze represented by the following gates in figure 1 can be simplified as in figure 2.

A classical algorithm may be capable of performing this simplification, thus reducing time complexity in a subset of quantumly-solved mazes.

Solution 2: Formulaic Representation

A formulaic representation maze could be achieved as follows (though many other formulaic methods are also possible):

Define a function $f(x, y, d)$, which takes two coordinates and a direction and outputs 0 or 1 (valid or invalid). Possibly, a function could have roots at only valid x, y, d values (i.e. the move is legal). If this function is simple to represent, then the maze is simple to represent, as a quick call to the function can determine whether a move is legal.

However, such a function seems unreasonable to create. Instead, we reframe the problem, classically labelling each valid move with a small integer. Then, the quantum algorithm will aim to move through each valid move, instead of through N/E/S/W directions. The goal may be to identify a function $g(m)$ that takes the move's integer, m, and outputs the starting/ending positions of the move. Although not identical, this technique is similar to "edge lists" in classical algorithms. As long as $g(m)$ can be identified such that g outputs the move's corresponding data (start position, end position, move direction), then each move's validity can be assessed efficiently. Such a solution may fit a more general class of functions and be easier to identify, in contrast to binary functions. Further studies should question if mathematical techniques known to show quantum supremacy or efficiency in quantum algorithms, such as factoring (Shor's Algorithm), can be used to optimize maze representation.

These solutions may serve especially useful for use cases under non-dynamic mazes (i.e. fewer variations in maze structures and more queries). If a user wishes to query a single maze (or graph) many times, a short series of checks or a simple mathematical formula could generate solutions significantly faster than classical algorithms (assuming the roadblock with probability amplification is also resolved). An example of such a

use case is delivery robots travelling through non-changing walkways. Of course, analogous optimizations exist in classical computers, and you theoretically could generate every possible path (at the cost of memory) to obtain $O(1)$ queries, but our work suggests that, assuming above challenges are overcome, there may exist a niche balance between preprocessing time, memory, and query time, where quantum supremacy is achievable.

Acknowledgments

The authors would like to express their sincere gratitude to the Lincoln Beaver Works Summer Institute (BWSI) and the Massachusetts Institute of Technology (MIT). Our participation in the BWSI Quantum Software Development program provided us with the foundational knowledge and skills in quantum computing that made this work possible. We extend our special thanks to the instructors and mentors of the program for their invaluable guidance, support, and encouragement throughout this research project.

References

[1] E. W. Dijkstra (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1(1), 269-271. doi:10.1007/BF01386390.

[2] L. K. Grover, A fast quantum mechanical algorithm for database search, in *Proceedings of the 28th Annual ACM Symposium on Theory of Computing* (1996), pp. 212-219, doi:10.1145/237814.237866.

[3] D. Goswami and N. Kumar. Quantum Algorithm to Solve a Maze: Converting the Maze Problem into a Search Problem. doi:10.48550/arXiv.1312.4116.

[4] A. M. Childs and J. Goldstone, *Spatial search by quantum walk*, doi:10.1103/PhysRevA.70.022314

[5] H. A. Priestley and M. P. Ward, *A multipurpose backtracking algorithm* *Journal of Symbolic Computation*, vol. 18, no. 1, pp. 1–40, July 1994, doi:10.1006/jsco.1994.1035.

[6] M. A. Nielsen and I. L.Chuang, *Quantum Computation and Quantum Information*. New York: Cambridge Univer- sity Press, 2000.

[7] D. Foead, A. Ghifari, M. Budi Kusuma, N. Hanafiah, E. Gunawan, *A Systematic Literature Review of A* Pathfinding*doi:10.1016/j.procs.2021.01.034